IMDb Movie Rating Scraper

Author: SANDHIYA R(Team 1)

Project Type:Individual project

PROJECT DESCRIPTION

The IMDb Movie Rating Scraper is a Python-based automation tool designed to extract movie-related information from the IMDb website. The scraper dynamically collects data such as movie title, release year, IMDb rating, and ranking from the IMDb Top 250 Movies list.

Since IMDb pages load content dynamically using JavaScript, Selenium WebDriver is used to automate browser actions and ensure accurate data extraction. The scraped data can be stored in a structured format and used for movie analysis, recommendation systems, and data science projects.

FEATURES

1. **Dynamic Movie Scraping**
   - Uses Selenium WebDriver to handle JavaScript-rendered content on IMDb.
2. **Top Movie Rankings**
   - Extracts movie title, release year, IMDb rating, and ranking from the Top 250 list.
3. **Structured Output**
   - Saves extracted data into a CSV file for easy access and analysis.
4. **Headless Mode (Optional)**
   - Can run without opening a browser window, improving performance.
5. **Expandable Design**
   - Can be extended to scrape additional movie details like genre, director, cast, and reviews.

TECHNOLOGIES USED

1. **Programming Language**
   - Python
2. **Libraries**
   - Selenium – for browser automation
   - pandas – for data storage and CSV export
   - time – for managing page load delays
   - webdriver_manager – for automatic ChromeDriver management
3. **Browser Automation**
   - Google Chrome with Selenium WebDriver
4. **Output Format**
   - CSV file

WORKING METHODOLOGY

1. Launch Chrome browser using Selenium WebDriver.
2. Load the IMDb Top 250 Movies webpage.
3. Wait for dynamic content to fully load.
4. Locate movie elements using XPath or CSS selectors.
5. Extract movie title, year, and IMDb rating.
6. Store the collected data into a pandas DataFrame.
7. Export the data into a CSV file.

SOURCE CODE

```python
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from webdriver_manager.chrome import ChromeDriverManager
import pandas as pd
import os
import time

options = webdriver.ChromeOptions()
options.add_argument("--start-maximized")
options.add_argument("--disable-blink-features=AutomationControlled")

driver = webdriver.Chrome(
    service=Service(ChromeDriverManager().install()),
    options=options
)

driver.get("https://www.imdb.com/chart/top/")
time.sleep(5)


driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
time.sleep(5)

movies = driver.find_elements(By.CSS_SELECTOR, "li.ipc-metadata-list-summary-item")

print("Movies found:", len(movies))

data = []

for i, movie in enumerate(movies, start=1):
    try:
        title = movie.find_element(By.CSS_SELECTOR, "h3.ipc-title__text").text
        year = movie.find_element(By.CSS_SELECTOR, "span.cli-title-metadata-item").text
        rating = movie.find_element(By.CSS_SELECTOR, "span.ipc-rating-star--rating").text
```

```
print("Movies found:", len(movies))

data = []

for i, movie in enumerate(movies, start=1):
    try:
        title = movie.find_element(By.CSS_SELECTOR, "h3.ipc-title__text").text
        year = movie.find_element(By.CSS_SELECTOR, "span.cli-title-metadata-item").text
        rating = movie.find_element(By.CSS_SELECTOR, "span.ipc-rating-star--rating").text
        data.append([i, title, year, rating])
    except:
        continue

driver.quit()

df = pd.DataFrame(data, columns=["Rank", "Movie Title", "Release Year", "IMDb Rating"])

print("Rows scraped:", len(df))
print(df.head())

desktop = os.path.join(os.path.expanduser("~"), "Desktop")
file_path = os.path.join(desktop, "imdb_top_250_movies.csv")
df.to_csv(file_path, index=False)

print("✅ CSV saved at:", file_path)
```

OUTPUT

```
Movies found: 250
Rows scraped: 250
   Rank                Movie Title Release Year IMDb Rating
0     1  The Shawshank Redemption         1994         9.3
1     2             The Godfather         1972         9.2
2     3           The Dark Knight         2008         9.1
3     4     The Godfather Part II         1974         9.0
4     5                12 Angry Men         1957         9.0
✅ CSV saved at: C:\Users\Sandhiya R\Desktop\imdb_top_250_movies.csv
PS C:\Users\Sandhiya R>
```

	A	B	C	D	E
1	Rank	Movie Title	Release Year	IMDb Rating	
2	1	The Shawshank Redemption	1994	9.3	
3	2	The Godfather	1972	9.2	
4	3	The Dark Knight	2008	9.1	
5	4	The Godfather Part II	1974	9	
6	5	12 Angry Men	1957	9	
7	6	The Lord of the Rings: The Return of the King	2003	9	
8	7	Schindler's List	1993	9	
9	8	The Lord of the Rings: The Fellowship of the Ring	2001	8.9	
10	9	Pulp Fiction	1994	8.8	

OUTCOMES

1. **Reliable Data Extraction**
   - Successfully scrapes dynamically loaded movie data.
2. **Up-to-Date Information**
   - Can be executed periodically to track rating changes.
3. **Data Analysis Ready**
   - CSV output can be used in dashboards, machine learning models, and reports.

APPLICATIONS

- Movie recommendation systems
- Trend analysis of movie ratings
- Data analytics and visualization
- Academic mini-projects

FUTURE ENHANCEMENTS

- Scraping movie reviews and performing sentiment analysis
- Adding filters for genre, year, or language
- Storing data in a database instead of CSV
- Integrating with machine learning models

CONCLUSION

In this project, a web scraping system was successfully developed to automatically collect movie details from the IMDb website. Using Python and Selenium, the system was able to handle dynamically loaded web content and extract important information such as movie title, release year, and IMDb rating. The scraped data was stored in a structured CSV file, making it easy to analyze and use for further applications. This project demonstrates the practical use of web scraping techniques in data analytics and shows how automated data collection can save time and improve accuracy.